

Scope delle Variabili

Scope delle Variabili

La visibilità e la durata delle variabili in C++.

Cos'è lo scope?

Immagina un edificio con stanze. Quello che metti in una stanza non è visibile dalle altre stanze. Se esci dalla stanza, le cose che hai lasciato lì scompaiono.

In C++, le variabili funzionano allo stesso modo: esistono solo nel "posto" in cui vengono dichiarate, e spariscono quando quel posto termina.

Questo "posto" si chiama **scope** (visibilità).

Le parentesi graffe creano lo scope

In C++, ogni coppia di `{ }` crea una nuova zona di codice con il proprio scope. Le variabili dichiarate dentro quella zona esistono solo lì:

```
int main() {  
    int x = 10;  
  
    {  
        int y = 20;    // y esiste solo in questo blocco interno  
        cout << x;     // OK - x è visibile anche qui  
        cout << y;     // OK - y è visibile qui  
    }  
  
    cout << x;         // OK - x esiste ancora  
    // cout << y; // ERRORE - y non esiste più, il blocco è finito  
}
```

Variabili locali: vivono dentro la funzione

Una **variabile locale** è dichiarata dentro una funzione. Esiste solo finché quella funzione è in esecuzione, e non è visibile dall'esterno:

```
void miaFunzione() {  
    int x = 10;    // x esiste solo dentro miaFunzione  
    cout << x;    // OK  
}  
  
int main() {  
    miaFunzione();  
    // cout << x;  // ERRORE - x non esiste qui  
    return 0;  
}
```

Ogni chiamata alla funzione crea una nuova copia delle variabili locali, che poi sparisce quando la funzione termina.

Variabili globali: visibili ovunque

Una **variabile globale** è dichiarata fuori da qualsiasi funzione. È visibile da tutto il codice nel file:

```
int contatore = 0;    // variabile globale - visibile ovunque  
  
void incrementa() {  
    contatore++;      // può accedere e modificare la variabile globale  
}  
  
int main() {  
    cout << contatore << endl;    // 0  
    incrementa();  
    incrementa();  
    cout << contatore << endl;    // 2  
    return 0;  
}
```

Usa le variabili globali con cautela. Il fatto che qualsiasi funzione possa modificarle le rende difficili da controllare nei programmi grandi.

Le variabili dei cicli

Le variabili dichiarate dentro un `for` esistono solo per la durata del ciclo:

```
for (int i = 0; i < 5; i++) {  
    cout << i << " ";    // OK  
}  
// cout << i;    // ERRORE - i non esiste più fuori dal ciclo
```

Questo è un vantaggio: il contatore `i` non inquina il resto del codice.

Shadowing: quando due variabili hanno lo stesso nome

Se dichiari una variabile locale con lo stesso nome di una globale, la locale "nasconde" la globale dentro il suo scope. Si chiama **shadowing**:

```
int x = 100;    // variabile globale  
  
void funzione() {  
    int x = 50;    // variabile locale - nasconde quella globale  
    cout << x;    // stampa 50 (usa la locale)  
}  
  
int main() {  
    cout << x;    // stampa 100 (usa la globale)  
    funzione();  
    cout << x;    // stampa 100 (la globale non è cambiata)  
    return 0;  
}
```

Evita lo shadowing: usa nomi diversi per variabili diverse. Il codice è molto più chiaro.

La variabile `static` : sopravvive tra le chiamate

Normalmente, una variabile locale ricomincia da zero ogni volta che la funzione viene chiamata. Con la parola chiave `static`, il valore viene conservato tra una chiamata e l'altra:

```

void contaChiamate() {
    int locale = 0;           // ricomincia da 0 ad ogni chiamata
    static int statica = 0;   // conserva il valore tra le chiamate

    locale++;
    statica++;

    cout << "Locale: " << locale << ", Statica: " << statica << endl;
}

int main() {
    contaChiamate(); // Locale: 1, Statica: 1
    contaChiamate(); // Locale: 1, Statica: 2
    contaChiamate(); // Locale: 1, Statica: 3
    return 0;
}

```

I parametri delle funzioni

I parametri di una funzione si comportano come variabili locali: esistono solo dentro la funzione.

```

void funzione(int a, int b) {
    // a e b sono locali a questa funzione
    int somma = a + b;
    cout << somma;
}

// a, b e somma non esistono qui

```

Consigli pratici

- **Dichiara le variabili vicino a dove le usi:** non in cima alla funzione se le usi solo alla fine.
- **Evita le variabili globali** quando possibile. Preferisci passare i valori come parametri.
- **Evita lo shadowing:** dai nomi diversi alle variabili in scope diversi.

```
// Meno chiaro: variabile dichiarata lontano dall'uso
int risultato;
// ... molte righe di codice ...
risultato = a + b;

// Più chiaro: dichiarazione e uso insieme
int risultato = a + b;
```